

AIKernel Phase-1 Paper 03: Pre-Inference Admissibility Governance

統治されたAI推論のための決定論的受理制御モデル

Author: Takuya Sogawa

ORCID: <https://orcid.org/0009-0009-7499-2595>

Version: v0.2.0

Publication date: 2026-05-25

DOI: <https://doi.org/10.5281/zenodo.20308639>

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Canonical language: English

Companion translation: Japanese

Series position: AIKernel / AIOS Phase-1 Specification Series, Part II-3: Governance Layer

Target: AIKernel.NET Governance / PreInference

Proposed namespace: AIKernel.Abstractions.Governance.PreInference

Stability: Experimental / Non-normative

Abstract

本稿は、AIKernel 実行パイプラインに推論要求を入れてよいかを、推論前に決定する **Pre-Inference Admissibility Governance**（推論前受理統治）を定義する。

本モデルは、確率的なLLM推論が開始される前に、入力妥当性、コンテキスト完全性、ケイパビリティ制約、破壊的副作用リスク、および計算複雑性を評価する。具体的には、Prompt Injection / Override Gate、Capability Admission Gate、Critical Operation Gate、Computational Complexity Gate から構成される多段ゲートアーキテクチャを導入する。

本稿の中心命題は、危険な推論を生成後に修復することではない。推論に入れてはならない要求は、推論開始前に停止・変形・委譲・分解・明確化されなければならない。したがって、推論前受理層は、ユーザー意図、統治されたコンテキスト、ケイパビリティ宣言、および確率的モデル実行の間に置かれる、fail-closed で replayable な決定論的境界である。

破壊的副作用制御と計算複雑性制御を分離することで、AIKernel は、不可逆な運用被害と計算論的な推論崩壊という二種類の失敗を、推論前に抑制する。本モデルは、Trajectory Governance、Tool Governance、および決定論的リプレイに接続するための形式的な受理制御基盤を提供する。

Keywords

AIKernel; AIOS; Pre-Inference Governance; Admissibility; Admission Control; Critical Operation Gate; Computational Complexity Gate; Capability Admission; Prompt Injection Defense; Fail-Closed; Deterministic Replay; ReplayLog; Policy Decision Point; Governed AI Inference

1. Introduction

LLMベースのシステムは、要求が本当に推論へ入ってよいかを確認しないまま推論を開始しがちである。通常のチャットアプリケーションでは、これは単なる低品質な回答で終わるかもしれない。しかし、ツール、ファイル、リポジトリ、外部API、OS的な資源へ接続されたAIエージェントでは、誤って受理された要求が、破壊的操作、権限誤用、コンテキスト汚染、あるいは推論崩壊へ直結する。

AIKernel は、推論の受理判定をカーネル責務として扱う。確率的モデルを呼び出す前に、そのタスクが安全であり、認可されており、有界であり、現在のProviderで扱うことが計算論的に適切であるかを、決定論的なゲート群によって検証しなければならない。

本稿は AIKernel / AIOS Phase-1 Specification Series の **Paper 03** である。Paper 01 は ROM 知識基盤を定義し、Paper 02 は VFS ストレージ境界とケイパビリティを持つセッションを定義する。Paper 03 は、それらを前提として、推論を開始してよいかを決める entry-control layer を定義する。

基本原則は次のとおりである。

確率的推論は、決定論的な受理判定が成功した後にのみ許可される。

2. Problem Statement

推論前リスクは、大きく二つに分けられる。

2.1 不可逆な破壊的副作用

LLM は、ファイル削除、永続データの上書き、スクリプト実行、権限変更、外部システムへの書き込みなど、不可逆な結果を伴う操作を要求される場合がある。このような要求が、明示的な意図、権限、スコープ、可逆性を確認する前にモデル生成やツール計画へ渡されると、システムは危険な行為を生成または実行してしまう可能性がある。

このリスクは、推論後のフィルタだけでは不十分である。推論へ入る前に検出されなければならない。

2.2 計算論的な推論崩壊

一部のタスクは、現在利用可能なモデルの推論能力や検証能力を超える。Sikka and Sikka は、TransformerベースのLLMおよびLLMベースエージェントが、一定以上の計算的・エージェント的複雑性を持つタスクにおいて、遂行や正確性検証に限界を持つことを論じている。AIKernel は、このような過剰な計算複雑性を、単なる実行時品質問題ではなく、推論前の受理問題として扱う。

例として、深い再帰、巨大な状態空間探索、検証困難なタスク、自己参照的なタスク、与えられた予算内では正しさを確認できない要求などがある。

2.3 不透明な受理判断

多くのシステムでは、単一のブラックボックス分類器やLLMプロンプトによって安全性判定を行う。しかし、この方式では判定が再現可能でなく、監査証跡も弱い。AIKernel は、決定論的Validator、バージョン管理されたPolicy、ReasonCode、ReplayRecordを要求する。

3. Design Goals

Pre-Inference Admissibility Governance は、次の設計目標を持つ。

- 決定論的な受理判定。** 同一入力、同一コンテキスト、同一Policy、同一Validator、同一Budgetは、同一の受理結果を返さなければならない。
- Fail-Closed。** Unknown、曖昧、または検証不能な状態は、拒否、明確化、または安全な形への変形へ倒す。
- コンテキスト完全性。** 検証済みで汚染されていないコンテキストのみが推論へ影響できる。
- ケイパビリティ整合性。** 要求された操作は、認可済みかつ物理的に利用可能な能力と整合していなければならない。
- 計算上の有界性。** モデル予算を超えるタスクは、委譲、分解、明確化、または拒否されなければならない。
- Replayability。** すべての受理判定は、不変の証拠から再構成可能でなければならない。

4. Scope and Boundary

Pre-Inference Governance は、**推論が始まる前** の entry-control mechanism である。

本稿は以下を目的としない。

- 候補アクションを生成すること
- 推論中の意味軌道を監視すること
- 生成後のツール候補を順位付けすること
- 最終回答の正しさを検証すること
- 事後安全性チェックを置き換えること
- 評価中に新しいケイパビリティを付与すること

Layer / Paper	Governance Role
Paper 01: ROM Format and Knowledge Snapshot Model	信頼可能な知識単位とスナップショット同一性を提供する。
Paper 02: VFS Architecture and Semantic Storage Model	ストレージ能力とセマンティックストレージ境界を提供する。
Paper 03: Pre-Inference Admissibility Governance	推論を開始してよいかを判定する。
Paper 04: Trajectory Governance Model	受理後の実行軌道、収束、候補リスクを監視する。

5. Pre-Inference Gate Architecture

受理判定パイプラインは、独立した決定論的ゲート群から構成される。



各ゲートは、最終的な判定を返すか、安全要件を付与する。推論が開始されるためには、すべてのValidatorが成功するか、要求が安全な状態へ変形されなければならない。

5.1 Prompt Injection / Override Gate

このゲートは、System Prompt、RootGoal、信頼済みコンテキスト、ポリシー、ケイパビリティ付与を上書きしようとする入力を拒否する。

AIKernel は、単なる意味理解だけに依存しない。信頼済み命令、ユーザー入力、検索されたコンテキスト、外部アーティファクトを異なる provenance domain に分離し、信頼されていない内容がポリシー、能力付与、実行権限を変更できないようにする。

5.2 Capability Admission Gate

このゲートは、要求された操作が現在のセッションで認可され、かつ利用可能な能力の範囲内にあるかを検証する。VFS や Provider 宣言から得られる capability facts を消費する。

利用できない権限を要求するタスクは、楽観的にモデルへ渡されない。拒否、明確化、委譲、または権限縮小された安全な形へ変形される。

5.3 Critical Operation Gate

Critical Operation Gate は、破壊的、永続的、特権的、または外部副作用を伴う操作を制御する。

5.4 Computational Complexity Gate

Computational Complexity Gate は、そのタスクを現在の LLM Provider へ受理してよいか、決定論的ソルバーへ委譲すべきか、分解すべきか、または計算複雑性が過剰で拒否すべきかを評価する。

6. Critical Operation Gate

このゲートの目的は、高度な言語理解そのものではない。目的は、破壊的、永続的、特権的、外部副作用を持ち得る操作を、推論前に保守的に分類することである。

6.1 Two-Stage Detection Pipeline

危険語が含まれるだけで拒否するのは過剰である。たとえば `rm -rf` が危険である理由を説明してほしい、という教育的要求は、実行要求とは異なる。そこで本ゲートは二段階で評価する。

Stage	Purpose
Trigger Detection	操作シグネチャ、対象ツール、インフラ能力、副作用マーカを検出する。
Intent Classification	説明、計画、実行、不明のいずれであるかを分類する。

6.2 Task Intent Classes

TaskIntent	Meaning	Governance policy
Unknown	意図を決定論的に特定できない。	Clarify または NoExecution によって fail closed する。
Explanation	概念説明のみを求めている。	ReadOnly または NoExecution として通過させる。
Plan	後続ターンで実行につながり得る手順立案を求めている。	DryRun、NoExecution、または明確化へ変形する。
Execution	実環境への直接作用を求めている。	確認、スナップショット、スコープ制限、または拒否を適用する。

6.3 Minimal Parameter Profile

状態爆発を避けるため、操作候補は最小の決定論的プロファイルへ縮約される。

Parameter	Values
OperationKind	Read, Write, Delete, Execute, NetworkWrite, PermissionChange
TargetScope	Unknown, Temporary, Workspace, Project, PersistentStore, ExternalSystem, System
Reversibility	Unknown, Reversible, SnapshotRequired, Irreversible
AuthorizationState	Unknown, ExplicitlyRequested, Ambiguous, NotRequested, Conflicting
CapabilityRisk	ReadOnly, Write, ExternalWrite, Privileged, Destructive

将来的には OperationScale (Single, Multiple, Recursive, Bulk, Unknown) の追加も考えられるが、初期モデルではプロファイルを最小限に保つ。

6.4 Disposition and Requirements

本ゲートは、基本方針と安全要件を分離する。

Disposition	Meaning
Permit	追加制約なしに許可する。
Transform	Requirements を付与したうえで許可する。
Clarify	曖昧性の解消をユーザーへ求める。
Deny	ポリシー違反または統治不能なリスクにより拒否する。

Requirement	Meaning
Confirmation	実行前に明示的な人間承認を要求する。
DryRun	変更を適用せず影響範囲をシミュレートする。
Snapshot	実行前にロールバックスナップショットを要求する。
ReadOnly	実行コンテキストを読み取り専用へ降格する。
RestrictScope	利用可能リソースを最小範囲へ制限する。
NoExecution	ツール実行と副作用を禁止する。
ReplayRequired	詳細な replay evidence を要求する。

7. Computational Complexity Gate

Computational Complexity Gate は、すべてのタスクを確率的言語モデルへ渡すべきではないという前提に立つ。タスクがモデル予算や検証能力を超える場合、カーネルは別の経路へルーティングする。

7.1 Task Complexity Profile

Field	Meaning
InputSizeEstimate	入力およびコンテキストサイズの推定値。
EstimatedCostClass	Trivial, Linear, Polynomial-small, Polynomial-large, Exponential, State-explosive, Verification-hard, Self-referential, Unbounded。
RecursionDepthEstimate	推論または分解に必要な深度の推定。
StateSpaceEstimate	追跡すべき状態空間の推定。
VerificationDifficulty	出力正当性の検証困難性。

7.2 Model Execution Budget

Field	Meaning
ContextWindow	利用可能な最大コンテキストウィンドウ。
OutputTokenBudget	最大生成予算。
ExternalSolverAvailability	SAT/SMT、証明支援系、SQLエンジン、静的解析器、ドメインリゾルバなどの決定論的ソルバー能力。

Field	Meaning
CalibratedErrorProfile	Providerのバージョン管理された履歴的エラープロファイル。

7.3 Complexity Decisions

Decision	Meaning
Permit	受理予算内であり推論へ入れてよい。
DelegateToSolver	LLMを迂回して決定論的ソルバーへ委譲する。
Decompose	大きすぎるが分割可能なタスクとして小タスクへ分解する。
Clarify	曖昧性によりプロファイル化できない。
Deny	無制限、安全でない、または検証不能である。

Permit は、LLMがタスクを正しく解けることの証明ではない。あくまで推論前
の入場許可であり、以降の Trajectory Governance や Tool Governance に
よる動的監視へ引き渡すことを意味する。

8. Formal Model

8.1 Admission Function

受理判定は次のように表される。

```
Admissible(request, context, capability_set, policy, budget)
-> Disposition + Requirements + Evidence
```

8.2 Initial State

初期評価状態は次のように定義される。

```
S0 = (input, context, capability_set, metadata, policy_version, budget)
```

評価中に、TaskIntent、CriticalOperationProfile、TaskComplexityProfile
が抽出される。

8.3 Transition

```
S0 --AdmissibilityCheck--> S1
```

S1 は、要求が受理、変形、委譲、分解、または明確化された場合にのみ存在す
る。それ以外の場合、遷移は Deny によって停止する。

8.4 Invariants

- コンテキストは内部整合的で改ざんされていない。
- ケイパビリティ主張は明示的かつ認可済みでなければならない。
- Unknown な Validator 状態は fail closed しなければならない。
- 破壊的副作用は受理前に発生してはならない。
- Complexity profile は決定論的Extractorによって生成されなければならない。

- すべての判定は replay evidence を出力しなければならない。

9. Replay and Audit Requirements

すべての受理判定は `AdmissibilityReplayRecord` として保存される。この Record は以下を含む。

- `final admissibility_decision`
- `critical operation result`
- `attached requirements`
- `task complexity profile`
- `model execution budget`
- `policy version`
- `validator versions`
- `reason code`
- `delegated solver identity, if any`
- `timestamp and trace identifier`

この Record の目的は、確率的モデル出力を再現することではない。推論が始まる前に、要求を許可、変形、委譲、分解、明確化、または拒否した決定論的判断を再構成することである。

10. Security Considerations

Pre-Inference Governance は、次のリスクを扱う。

Risk	Governance response
Prompt injection	信頼済み Policy domain と非信頼 Context を分離する。
Capability escalation	宣言または認可された能力を超える要求を拒否する。
Destructive execution	確認、スナップショット、DryRun、スコープ制限、または拒否を適用する。
Ambiguous intent	明確化を要求するか、NoExecution 制約を付与する。
Computational overload	推論前に委譲、分解、明確化、または拒否する。
Non-replayable admission	決定論的証拠と ReasonCode を保存する。

11. Implementation Mapping

AIKernel.NET では、本モデルを次のような interface contracts へ写像できる。

```
namespace AIKernel.Abstractions.Governance.PreInference;

public interface IPreInferenceGovernancePipeline
{
    ValueTask<PreInferenceResult> ProcessAsync(
```



```

        IContextSnapshot context,
        CancellationToken cancellationToken = default);
    }

    public interface ICriticalOperationGate
    {
        ValueTask<CriticalOperationGateResult> EvaluateAsync(
            CriticalOperationProfile profile,
            CancellationToken cancellationToken = default);
    }

    public interface IComputationalComplexityGate
    {
        ValueTask<ComplexityGateResult> EvaluateAsync(
            TaskComplexityProfile taskProfile,
            ModelExecutionBudget budget,
            CancellationToken cancellationToken = default);
    }

```

DTOの詳細は実装成果物である。アーキテクチャ上の要件は、それらが immutable、versioned、serializable、replayable であることである。

12. Limitations

1. **構造化コンテキストへの依存**。本モデルは、コンテキスト断片が quarantine と構造検証を通過していることを前提とする。
2. **決定論的Validatorへの依存**。受理判定は確率的LLM呼び出しに依存してはならず、決定論的コード、ポリシールール、スキーマ、またはバージョン管理されたヒューリスティックでなければならない。
3. **タスク抽出は全知ではない**。Task profile は ground truth ではなく governance evidence である。決定論的に導出できない場合、Extractor は Unknown を返し、pipeline は fail closed しなければならない。
4. **軌道統治は扱わない**。本稿は推論開始可否を決定する。Runtime convergence、semantic drift、tool-candidate risk、post-generation control は Trajectory Governance の責務である。
5. **完全なpolicy algebraではない**。ABAC や XACML 的なポリシー評価は PDP 層で利用され得るが、本稿は完全な企業向け認可言語ではなく、受理境界と証拠モデルを定義する。

13. Relationship to Other Phase-1 Papers

- **Paper 01** は、信頼済みコンテキスト入力として利用される ROM 知識とスナップショット同一性を提供する。
- **Paper 02** は、Capability Admission に利用されるストレージ能力情報とセマンティックストレージ境界を提供する。
- **Paper 04** は、受理後の意味軌道を監視する。
- **Paper 05 以降** は、AIKernel 実行パイプラインの一部として受理判定を消費する。

14. Conclusion

Pre-Inference Admissibility Governance は、確率的推論を事後的に安全にするものではない。推論へ入れてはならない要求が開始されることを防ぐ。

Prompt Injection 防御、Capability Admission、Critical Operation Control、Computational Complexity Gate を組み合わせることで、AIKernel は統治されたAI実行のための決定論的な入口境界を確立する。その結果、確率的推論の開始は、fail-closed、replayable、auditable なシステム遷移として扱われる。

References

1. Hartmanis, Juris, and Richard E. Stearns. “On the Computational Complexity of Algorithms.” Transactions of the American Mathematical Society, vol. 117, 1965, pp. 285-306. DOI: 10.1090/S0002-9947-1965-0170805-7.
2. Sikka, Varin, and Vishal Sikka. “Hallucination Stations: On Some Basic Limitations of Transformer-Based Language Models.” arXiv:2507.07505, 2025. DOI: 10.48550/arXiv.2507.07505.
3. National Institute of Standards and Technology. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, 2014. DOI: 10.6028/NIST.SP.800-162.
4. OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 22 January 2013.
5. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
6. Sogawa, Takuya. “AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20306539.
7. Sogawa, Takuya. “AIKernel Phase-1 Paper 02: VFS Architecture and Semantic Storage Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20307891.